1. Estructura del repositorio y ramas

Dos ramas: **proceso** (trabajo en curso) y **main** (versión estable). Todo el grupo trabaja sobre la misma rama. Sugerencias para ordenarnos:

- **main** solo recibe lo que ya fue probado ejecutando todo desde cero (BD borrada, scripts en orden, tests sin errores inesperados).
- Commits chicos, con mensaje descriptivo (ej: "Agrega SP de alta de partido con validaciones").

Estructura propuesta

```
repo/
  README.md / Caratula
  doc/                  <- PDFs, nomenclatura, imágenes
  DER/                  <- diagrama (PNG/JPG)
  Tablas/               <- base, tablas, SP (ABM y negocio) y tests, con su orden
  Datos/                <- datos de prueba
  Importacion/          <- Entrega 6 (separado de todo lo demás)
```

Por ahora Tablas concentra tablas, SP y tests para mayor orden; la organización se puede cambiar más adelante.

2. Norma de nombres de archivos

Formato: `NN_Tipo_Nombre.sql`, donde NN son los dígitos de orden de ejecución. La numeración es global y se lleva dentro de la carpeta. Todos los archivos son **.sql** (en la captura los archivos aparecen como .txt: hay que renombrarlos).

Tipo	Uso	Ejemplo
Base	Crear BD y esquemas	00_Base_Mundial.sql
Tabla	Creación de tablas y restricciones	01_Tabla_Mundial.sql
Sp	SP de ABM o de negocio	02_Sp_Partido.sql
Test	Pruebas (nombre alineado al script probado)	03_Test_Tipo_Sp_Partido.sql

- El orden debe respetar dependencias: Base, Tablas, SP, Testing.
- Los tests van en archivos separados de los de creación. Cada test exitoso y de validación debe indicar en comentarios el resultado esperado.

3. Norma de nomenclatura de objetos (a documentar)

El enunciado exige definir y documentar una norma para nombres de tablas, SP y variables, e incluirla en el documento final. Proponemos fijarla ahora y escribirla en `doc/Nomenclatura.pdf`. Borrador para decidir entre todos:

Objeto	Convención sugerida	Ejemplo
Esquemas	minúsculas, por módulo	sedes, equipos, partidos, arbitros, publicidad
Tablas	PascalCase, singular	Partido, Seleccion, Jugador
PK / FK	PK_Tabla / FK_Tabla_Referenciada	PK_Partido, FK_Partido_Sede
UNIQUE / CHECK / DEFAULT	UQ_ / CK_ / DF_ + tabla + columna	UQ_Jugador_Dorsal
SP ABM	esquema.Tabla_Operacion (Alta, Baja, Modificacion)	partidos.Partido_Alta
SP negocio	esquema.Verbo_Proceso	partidos.Registrar_Resultado
Variables y parámetros	camelCase con prefijo @	@idPartido, @minutoGol
Cursores	crs + propósito	crsConvocados

4. Reglas de todos los scripts

- ☐ Comentario inicial: universidad, materia, integrantes del grupo, fecha y objetivo del script.
- ☐ Validar existencia antes de crear o borrar (`IF OBJECT_ID(...) IS NULL`, `DROP ... IF EXISTS` o `CREATE OR ALTER`).
- ☐ Re-ejecutables: correr dos veces el mismo script no debe romper nada.
- ☐ Sin SQL dinámico, sin CLR, sin herramientas externas.
- ☐ Manejo de errores con TRY/CATCH y THROW; en transacciones usar SET XACT_ABORT ON y ROLLBACK en el CATCH.
- ☐ Mensaje de validación único: acumular todos los errores del SP en una variable y lanzar un solo THROW al final.
- ☐ Los scripts de importación se mantienen separados de los demás.

5. Checklist de cierre de la Entrega 5

- ☐ Todos los archivos .sql, con numeración de orden y comentario de cabecera.
- ☐ Script de Base y esquemas, y script de Tablas con restricciones.
- ☐ SP de ABM de todas las tablas.
- ☐ Al menos 10 validaciones con mensaje único agrupado.
- ☐ SP de negocio transaccionales en scripts separados.
- ☐ Un test exitoso y un test de validaciones por cada script de SP, con resultado esperado comentado.
- ☐ Uso de cursores donde corresponda; sin SQL dinámico ni CLR.
- ☐ Prueba completa: borrar la BD, ejecutar todo en orden, sin errores no esperados.
- ☐ Todos los integrantes con commits propios en el repositorio.
- ☐ Merge de proceso a main solo con todo lo anterior validado.
- ☐ Todos listos para exponer y responder en el coloquio (ejecución en vivo, todos participan).